

50 Python Interview Questions & Answers

Basic → Intermediate → Advanced

A structured fresher-to-professional prep guide for
AI/ML & Generative AI Engineer Interviews

Prepared for: Sangam Kumar Gupta
B.Tech CSE (Data Science) • NIET, Greater Noida

BASIC LEVEL (Q1 - Q18)

Q1. What is Python and what are its key features?

Python is a high-level, interpreted, general-purpose programming language known for its readability and simplicity. Key features: dynamically typed, interpreted (not compiled to machine code directly), supports multiple paradigms (OOP, functional, procedural), has automatic memory management (garbage collection), a huge standard library, and is platform-independent.

Q2. What is the difference between a list and a tuple?

Lists are mutable (can be changed after creation) and defined with square brackets `[]`; tuples are immutable and defined with parentheses `()`. Because tuples are immutable, they are slightly faster and can be used as dictionary keys or set elements, while lists cannot. Use a list when data changes over time, and a tuple for fixed collections like coordinates.

```
lst = [1, 2, 3]      # mutable
tup = (1, 2, 3)      # immutable
lst[0] = 99          # OK
# tup[0] = 99        # TypeError
```

Q3. What is the difference between `==` and `is` in Python?

`==` compares values (checks if two objects have the same content), while `is` checks identity (whether both references point to the exact same object in memory). Two equal-value objects can still be different objects, so use `is` only for identity checks like `x is None`.

```
a = [1, 2, 3]
b = [1, 2, 3]
print(a == b)  # True (same values)
print(a is b)  # False (different objects)
```

Q4. What are mutable and immutable data types in Python?

Mutable objects can be changed after creation without changing their identity: **list**, **dict**, **set**, **bytearray**. Immutable objects cannot be changed once created; any 'modification' creates a new object: **int**, **float**, **str**, **tuple**, **frozenset**, **bool**. This matters for function arguments, hashing, and avoiding unexpected side effects when objects are shared.

Q5. What are `*args` and `**kwargs` used for?

`*args` lets a function accept any number of positional arguments, collected as a tuple. `**kwargs` lets it accept any number of keyword arguments, collected as a dictionary. They are commonly used for writing flexible, generic functions and wrapper/decorator functions.

```
def demo(*args, **kwargs):
    print(args)      # (1, 2, 3)
    print(kwargs)    # {'x': 10}

demo(1, 2, 3, x=10)
```

Q6. What is the difference between a shallow copy and a deep copy?

A shallow copy (**copy.copy()**) creates a new outer object but nested objects are still shared references. A deep copy (**copy.deepcopy()**) recursively copies every nested object, so the copy is fully independent from the original.

```
import copy
original = [[1, 2], [3, 4]]
shallow = copy.copy(original)
deep = copy.deepcopy(original)

shallow[0][0] = 99
print(original[0][0]) # 99 -> nested list shared

deep[1][0] = 100
print(original[1][0]) # 3 -> fully independent
```

Q7. What are list comprehensions and why use them?

A list comprehension is a concise syntax to build a list in one line by looping over an iterable and optionally filtering it. They are generally faster and more readable than an equivalent for-loop with `.append()`.

```
squares = [x**2 for x in range(10) if x % 2 == 0]
# [0, 4, 16, 36, 64]
```

Q8. What is the difference between a function and a method?

A function is a standalone block of reusable code defined with **def**, called independently. A method is a function that is defined inside a class and is bound to an object; it is called on an instance and automatically receives **self** as its first argument.

Q9. What is the purpose of the self keyword?

self refers to the current instance of a class. It lets each method access and modify the instance's own attributes and call its own methods. It's the first parameter of every instance method, though Python passes it automatically when you call **obj.method()**.

```
class Employee:
    def __init__(self, name):
        self.name = name
    def greet(self):
        return f"Hi, I am {self.name}"
```

Q10. What is the difference between a local variable and a global variable?

A local variable is defined inside a function and only accessible within that function's scope. A global variable is defined at the module level and accessible everywhere, though inside a function you must declare **global varname** to modify it rather than just read it.

Q11. What are Python's built-in data types?

Numeric: **int**, **float**, **complex**. Sequence: **str**, **list**, **tuple**, **range**. Mapping: **dict**. Set types: **set**, **frozenset**. Boolean: **bool**. Binary: **bytes**, **bytearray**, **memoryview**. There is also **NoneType** for the null value **None**.

Q12. What is the difference between `append()` and `extend()` in a list?

`append()` adds its argument as a single element to the end of the list, even if it's a list itself. **`extend()`** iterates over its argument and adds each element individually.

```
lst = [1, 2]
lst.append([3, 4])    # [1, 2, [3, 4]]

lst2 = [1, 2]
lst2.extend([3, 4])   # [1, 2, 3, 4]
```

Q13. What is the difference between a dictionary and a set?

A dictionary stores key-value pairs where keys are unique and used to look up values. A set stores only unique unordered values with no associated values. Both use hashing internally, giving average $O(1)$ lookup, insertion, and deletion.

Q14. What is string slicing? Give an example.

Slicing extracts a substring/sub-sequence using **`[start:stop:step]`** notation. Start is inclusive, stop is exclusive, and step defines direction/increment. Negative indices count from the end.

```
s = "Interview"
print(s[0:4])      # 'Inte'
print(s[-4:])      # 'view'
print(s[::-1])     # 'weivretnI' (reversed)
```

Q15. What is the difference between `range()` and `xrange()`?

In Python 2, **`range()`** returned a list (loaded fully into memory) while **`xrange()`** returned a lazy generator-like object. In Python 3, **`xrange()`** was removed and **`range()`** itself now behaves like the old `xrange()` -- it returns a memory-efficient range object that generates values on demand.

Q16. What are Python's key OOP principles?

Encapsulation (bundling data and methods, restricting direct access using naming conventions like **`__protected`** or **`__private`**), Inheritance (a class reusing/extending another class's behaviour), Polymorphism (same interface/method name behaving differently across classes), and Abstraction (hiding implementation details behind a simple interface, often via the **`abc`** module).

Q17. What is exception handling and how is it done in Python?

Exception handling lets a program gracefully respond to runtime errors instead of crashing, using **`try / except / else / finally`**. Code that might fail goes in **`try`**, the recovery logic goes in **`except`**, code that runs only if no exception occurred goes in **`else`**, and cleanup code that always runs goes in **`finally`**.

```
try:
    result = 10 / 0
except ZeroDivisionError as e:
    print("Cannot divide by zero:", e)
else:
    print("Success")
finally:
    print("Always runs")
```

Q18. What is the difference between `remove()`, `pop()`, and `del`?

`remove(value)` deletes the first matching value from a list. **`pop(index)`** removes and returns the element at that index (default last), useful when you need the removed value. **`del list[index]`** deletes the element at an index (or the whole list) without returning it.

INTERMEDIATE LEVEL (Q19 - Q38)

Q19. What are decorators in Python?

A decorator is a function that wraps another function or class to extend/modify its behaviour without changing its source code. It takes a function as input and returns a new function, typically applied using the **@decorator_name** syntax above a function definition. Common uses: logging, timing, authentication checks, caching.

```
def log_calls(func):
    def wrapper(*args, **kwargs):
        print(f"Calling {func.__name__}")
        return func(*args, **kwargs)
    return wrapper

@log_calls
def add(a, b):
    return a + b

add(2, 3) # prints 'Calling add' then returns 5
```

Q20. What are generators and how are they different from normal functions?

A generator is a function that uses **yield** instead of **return** to produce a sequence of values lazily, one at a time, pausing its state between calls. Unlike a normal function that computes and returns everything at once, a generator is memory-efficient because it doesn't store the whole sequence in memory -- ideal for large or infinite data streams.

```
def count_up_to(n):
    i = 1
    while i <= n:
        yield i
        i += 1

for num in count_up_to(5):
    print(num) # 1 2 3 4 5, one at a time
```

Q21. What is the difference between an iterator and an iterable?

An iterable is any object capable of returning its members one at a time, implementing **__iter__()** (e.g. list, tuple, dict). An iterator is the object actually produced by calling **iter()** on an iterable; it implements both **__iter__()** and **__next__()** and remembers its position. Every iterator is an iterable, but not every iterable is an iterator.

```
nums = [1, 2, 3]          # iterable
it = iter(nums)           # iterator
print(next(it))           # 1
print(next(it))           # 2
```

Q22. What are lambda functions? When would you use one?

A lambda is a small, anonymous, single-expression function defined with the **lambda** keyword instead of **def**. It's useful for short, throwaway functions passed as arguments to functions like **sorted()**, **map()**, or **filter()**, where defining a full named function would be unnecessary overhead.

```
students = [("Amit", 82), ("Riya", 91), ("Kunal", 76)]
students.sort(key=lambda s: s[1], reverse=True)
# sorted by score descending
```

Q23. Explain map(), filter(), and reduce().

map(func, iterable) applies a function to every element and returns an iterator of results. **filter(func, iterable)** keeps only elements for which the function returns True. **reduce(func, iterable)** (from **functools**) cumulatively applies a function to reduce the iterable to a single value.

```
from functools import reduce
nums = [1, 2, 3, 4]
squares = list(map(lambda x: x**2, nums))      # [1,4,9,16]
evens = list(filter(lambda x: x % 2 == 0, nums)) # [2,4]
total = reduce(lambda a, b: a + b, nums)      # 10
```

Q24. What is a closure in Python?

A closure is a nested (inner) function that remembers and has access to variables from its enclosing (outer) function's scope, even after the outer function has finished executing. It requires a nested function referencing a variable from the enclosing scope, and the outer function returning that inner function.

```
def make_multiplier(n):
    def multiplier(x):
        return x * n
    return multiplier

times3 = make_multiplier(3)
print(times3(10)) # 30, 'n' is remembered
```

Q25. What is the difference between @staticmethod, @classmethod, and instance methods?

An instance method takes **self** and can access/modify instance state. A **@classmethod** takes **cls** instead of **self** and works with the class itself (often used for alternate constructors). A **@staticmethod** takes neither **self** nor **cls** -- it's just a regular function placed inside a class for organizational purposes.

```
class Person:
    count = 0
    def __init__(self, name):
        self.name = name
        Person.count += 1

    @classmethod
    def total(cls):
        return cls.count

    @staticmethod
    def is_valid_name(name):
        return isinstance(name, str) and len(name) > 0
```

Q26. What is monkey patching in Python?

Monkey patching means dynamically modifying or extending a class or module at runtime -- for example, adding a method to a class after it has already been defined. It's powerful for testing (mocking) but risky in production code since it can make behaviour hard to trace.

```
class MyClass:
    def greet(self):
        return "Hello"

def new_greet(self):
    return "Hi there!"

MyClass.greet = new_greet    # monkey patch
print(MyClass().greet())    # 'Hi there!'
```

Q27. What is the Global Interpreter Lock (GIL)?

The GIL is a mutex in CPython that allows only one thread to execute Python bytecode at a time, even on multi-core processors. It simplifies memory management (reference counting) but means multi-threading doesn't give true parallelism for CPU-bound tasks in Python. For CPU-bound work, use **multiprocessing** (separate processes, separate memory, no GIL contention); for I/O-bound work, threading or asyncio still helps because the GIL is released during I/O waits.

Q28. What is the difference between multithreading and multiprocessing?

Multithreading runs multiple threads within a single process, sharing the same memory space -- good for I/O-bound tasks but limited by the GIL for CPU-bound tasks. Multiprocessing runs separate processes with their own memory and interpreter, giving true parallelism across CPU cores, ideal for CPU-bound tasks, at the cost of higher memory usage and inter-process communication overhead.

Q29. What are context managers and the with statement used for?

A context manager manages setup and teardown of a resource (like a file or a lock) automatically. The **with** statement calls `__enter__()` on entry and guarantees `__exit__()` is called on exit -- even if an exception occurs -- so resources are always released properly (e.g. a file is always closed).

```
with open("data.txt", "r") as f:
    content = f.read()
# file is automatically closed here, even on error
```

Q30. How do you create a custom context manager?

Either by implementing `__enter__` and `__exit__` methods on a class, or more simply using the `@contextmanager` decorator from **contextlib** with a generator function that has a single **yield** splitting setup code from teardown code.

```
from contextlib import contextmanager

@contextmanager
def timer():
    import time
    start = time.time()
    yield
    print(f"Elapsed: {time.time()-start:.2f}s")

with timer():
    sum(range(1000000))
```

Q31. What is the difference between deep learning and machine learning? (often asked to freshers with AI/ML background)

Machine learning is a broader field where algorithms learn patterns from data using techniques like decision trees, SVMs, or regression -- often requiring manual feature engineering. Deep learning is a subset of ML that uses multi-layer neural networks to automatically learn hierarchical features directly from raw data (images, text, audio), typically needing more data and compute but achieving stronger results on complex, unstructured data.

Q32. What is duck typing in Python?

Duck typing means Python cares about what an object can do (which methods/attributes it supports) rather than its actual type -- "if it walks like a duck and quacks like a duck, it's a duck." This is why Python doesn't require explicit interfaces; any object with a compatible `.read()` method, for instance, can be used where a file-like object is expected.

Q33. What is the difference between `__str__` and `__repr__`?

`__str__` returns a readable, user-friendly string representation of an object (used by `print()` and `str()`).

`__repr__` returns an unambiguous, developer-facing representation, ideally one that could recreate the object; it's used by the interpreter and when `__str__` isn't defined, Python falls back to `__repr__`.

```
class Point:
    def __init__(self, x, y):
        self.x, self.y = x, y
    def __str__(self):
        return f"({self.x}, {self.y})"
    def __repr__(self):
        return f"Point(x={self.x}, y={self.y})"
```

Q34. What are Python's magic/dunder methods? Give examples.

Dunder (double underscore) methods let custom classes integrate with Python's built-in syntax and functions. Examples: `__init__` (constructor), `__len__` (used by `len()`), `__add__` (used by `+`), `__eq__` (used by `==`), `__getitem__` (indexing/slicing), `__iter__` / `__next__` (iteration).

Q35. How does Python manage memory?

Python uses automatic memory management via a private heap. Every object has a reference count, and when that count drops to zero, the memory is immediately freed. A separate cyclic garbage collector (the `gc` module) periodically detects and cleans up reference cycles (e.g. two objects referencing each other) that reference counting alone can't catch.

Q36. What is the difference between a module and a package?

A module is a single `.py` file containing Python code (functions, classes, variables) that can be imported. A package is a directory containing multiple modules along with an `__init__.py` file, which groups related modules into a namespace/hierarchy.

Q37. What is PEP 8, and why does it matter?

PEP 8 is Python's official style guide, covering naming conventions, indentation (4 spaces), line length, whitespace usage, and import ordering. Following it makes code consistent and readable across teams, which is especially important for freshers to demonstrate professional coding habits in interviews and real projects.

Q38. What is the purpose of `__init__.py`?

`__init__.py` marks a directory as a Python package (required in older Python versions; optional but still common in Python 3 for namespace packages) and can run initialization code for the package or define what gets exposed when the package is imported.

ADVANCED LEVEL (Q39 - Q50)

Q39. What are metaclasses in Python?

A metaclass is 'the class of a class' -- it defines how classes themselves are constructed, just as a class defines how instances are constructed. By default, classes are instances of **type**. Custom metaclasses (by subclassing **type**) let you control class creation, e.g. auto-registering subclasses or enforcing coding standards. They're powerful but rarely needed in everyday application code -- frameworks like Django/SQLAlchemy use them internally.

```
class Meta(type):
    def __new__(cls, name, bases, dct):
        print(f"Creating class {name}")
        return super().__new__(cls, name, bases, dct)

class MyClass(metaclass=Meta):
    pass
# prints: Creating class MyClass
```

Q40. What are descriptors in Python?

A descriptor is an object that customizes attribute access on another class by implementing `__get__`, `__set__`, or `__delete__`. Python's own **property**, methods, and **staticmethod/classmethod** are all implemented using descriptors. They're used to add validation or computed behaviour when an attribute is accessed/assigned.

```
class PositiveNumber:
    def __set_name__(self, owner, name):
        self.name = "_" + name
    def __get__(self, obj, objtype=None):
        return getattr(obj, self.name)
    def __set__(self, obj, value):
        if value < 0:
            raise ValueError("Must be positive")
        setattr(obj, self.name, value)

class Product:
    price = PositiveNumber()
```

Q41. What is the difference between async/await and traditional threading?

async/await (asyncio) uses a single-threaded event loop with cooperative multitasking: coroutines voluntarily yield control (via **await**) during I/O waits, avoiding thread context-switching overhead entirely. Threading uses OS-level preemptive multitasking across multiple threads. Asyncio is very efficient for thousands of concurrent I/O-bound tasks (e.g. API calls) with low overhead, while threading/multiprocessing better suit blocking libraries or CPU-bound work respectively.

```
import asyncio

async def fetch(id):
    await asyncio.sleep(1)
    return f"data-{id}"

async def main():
    results = await asyncio.gather(fetch(1), fetch(2), fetch(3))
    print(results)

asyncio.run(main())
```

Q42. How does Python's garbage collector handle reference cycles?

Reference counting alone can't free two or more objects that reference each other (a cycle), since their counts never reach zero. Python's generational garbage collector periodically scans for such unreachable cycles using a mark-and-sweep-like algorithm and reclaims them. Objects are grouped into three generations based on age, since younger objects are collected more frequently than long-lived ones (generational hypothesis).

Q43. What is method resolution order (MRO) in Python?

MRO defines the order in which base classes are searched when looking up a method or attribute in multiple inheritance. Python uses the C3 linearization algorithm, which can be inspected via **ClassName.__mro__** or **ClassName.mro()**. It ensures a consistent, predictable order even in complex 'diamond' inheritance hierarchies.

```
class A:
    def hello(self): return "A"
class B(A):
    def hello(self): return "B"
class C(A):
    def hello(self): return "C"
class D(B, C):
    pass

print(D().hello())      # 'B'
print(D.__mro__)        # D -> B -> C -> A -> object
```

Q44. What is the difference between deepcopy and pickling for object serialization?

deepcopy creates an independent in-memory duplicate of an object, useful within the same running program. **pickle** serializes an object into a byte stream that can be saved to disk or sent over a network, and later deserialized (unpickled) -- even in a completely different process or machine. Pickle is for persistence/transport; deepcopy is for in-memory independence.

Q45. What are Python's memory-efficient alternatives to lists for large datasets?

generators (lazy evaluation, no full storage), **array** module (typed, compact arrays vs generic lists), **numpy** arrays (contiguous memory, vectorized operations, much less overhead per element than a Python list of objects), and **collections.deque** for efficient appends/pops from both ends.

Q46. Explain the difference between deep learning frameworks like TensorFlow and PyTorch (common GenAI/ML fresher question).

TensorFlow historically used static computation graphs (define-then-run) and now supports eager execution too; it's strong for production deployment (TensorFlow Serving, TF Lite, TF.js). PyTorch uses dynamic computation graphs (define-by-run), which feel more Pythonic and easier to debug, making it very popular in research and increasingly in production via TorchServe. Both support GPU acceleration and autograd for automatic differentiation.

Q47. What is a Python virtual environment, and why is it important?

A virtual environment is an isolated Python environment with its own installed packages and interpreter, separate from the system-wide Python installation. It prevents dependency conflicts between projects (e.g. Project A needing LangChain 0.1 and Project B needing LangChain 1.3) and makes projects reproducible. Created with **python -m venv env_name** and activated before installing project-specific packages with pip.

Q48. What is the difference between a synchronous and an asynchronous function call, using an API request example?

A synchronous call blocks execution -- the program waits idle until the API responds before moving to the next line. An asynchronous call lets the program continue other work (or handle other requests) while waiting for the response, resuming via a callback, future, or **await** once the result is ready. This is why async is preferred for I/O-heavy apps like web servers or apps calling multiple LLM/GenAI APIs concurrently.

Q49. What is the difference between REST API and gRPC (often asked for backend-adjacent ML roles)?

REST APIs use HTTP/JSON, are human-readable, widely supported, and simpler to debug -- ideal for public-facing APIs. gRPC uses HTTP/2 with Protocol Buffers (binary format), giving faster serialization and lower latency, and supports streaming natively -- often preferred for internal microservice-to-microservice communication where performance matters more than human readability.

Q50. How would you optimize a slow Python function? Walk through your approach.

A structured approach: (1) **Profile first** using **cProfile** or **timeit** to find the actual bottleneck instead of guessing. (2) Check algorithmic complexity -- an $O(n^2)$ solution often needs an $O(n \log n)$ or $O(n)$ rewrite (e.g. using a set/dict for $O(1)$ lookups instead of scanning a list). (3) Use built-in functions and libraries like **numpy** which are implemented in C and much faster than pure Python loops. (4) Avoid unnecessary object creation/copies inside loops. (5) For I/O-bound bottlenecks, use **async/threading**; for CPU-bound ones, consider **multiprocessing** or **vectorization**. (6) Only after all this, consider caching (**functools.lru_cache**) for repeated expensive calls.

```
from functools import lru_cache

@lru_cache(maxsize=None)
def fib(n):
    if n < 2:
        return n
    return fib(n-1) + fib(n-2)
```